

Backend From First Principles:

Sessions, Cookies and JWTs:

	Cookie	Session	JWT
What is it?	Browser storage/transport	Server-side authentication state	Signed token
Where state lives?	Client	Server	Client/token
Stateful?	Depends	Yes	Generally no
Verification	Browser sends it	Session-store lookup	Signature + claims verification
Automatically sent?	Yes , to matching domain/path	Through cookie	Only if stored in cookie; not automatic if Authorisation header
Main benefit	Convenient browser persistence/transport	Easy revocation & server control	Scalable distributed authentication

JWT = HEADER.PAYLOAD.SIGNATURE -> (JWT is encoded, not encrypted).

	Encoding	Encryption
Purpose	Change data into another format	Hide/protect data
Can anyone reverse it?	✅ Yes	❌ No, without the key
Requires secret key?	❌ No	✅ Yes
Provides confidentiality?	❌ No	✅ Yes
Example	Base64	AES, RSA

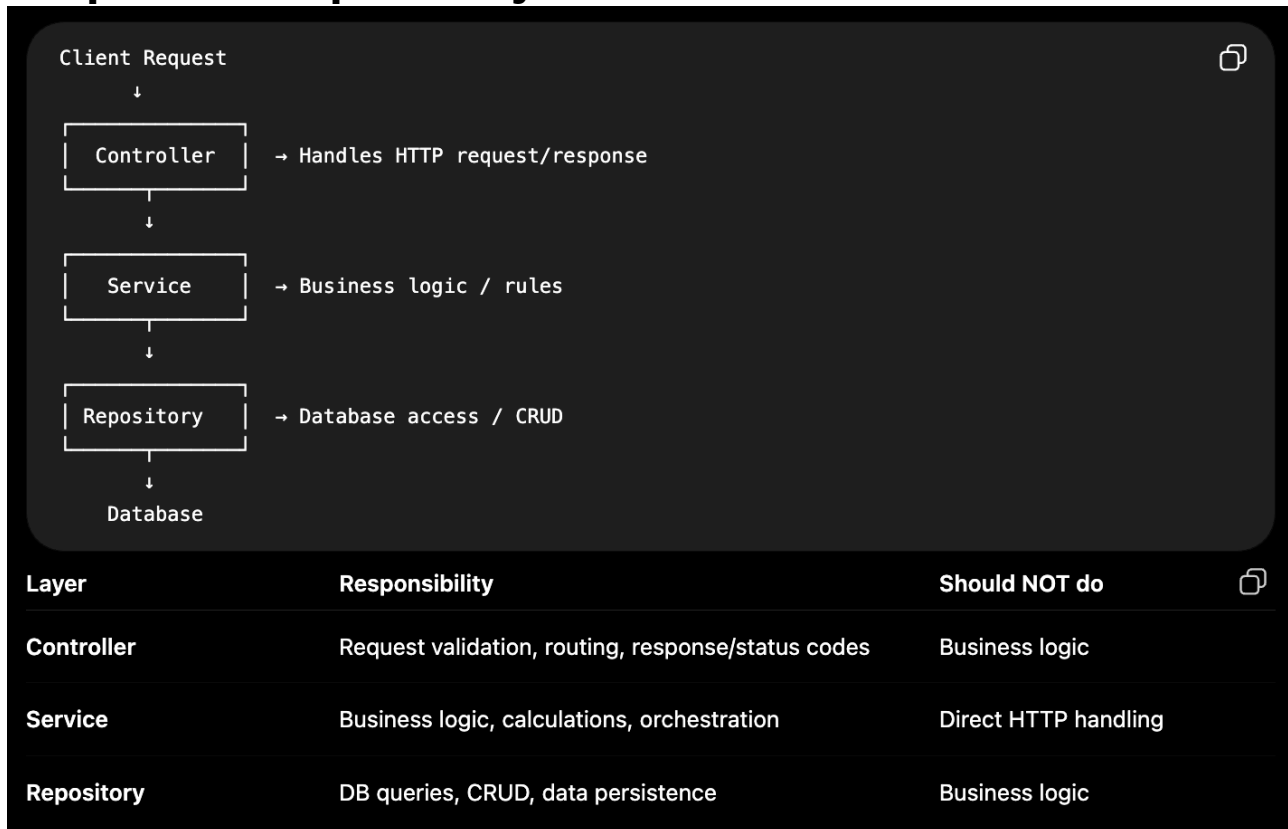
Problem with session less authentication:

If we want to revoke log in of all users in case of any situation we cannot. That's why we make stateless authentication short lived and with refresh tokens. On the other hand benefit is distributed/scalable systems.

Types of Authentications:

1. Stateful - { SessionId: User } stored in redis and SessionId sent to client to store in cookie.
2. Stateless - Signed JWT is returned to client, sends in all requests as header.
3. API Key - Generate an hashed Key and we can access server using this key. (for machine-machine interaction)
4. OAuth - Open Authorisation was introduced for providing access to resources of app 1 to app 2 -> sharing CONTROLLED access across platforms, Solution: Make tokens that have controlled permissions. This is OAuth but this did not provide authentication so OIDC(Open ID Connect) did that.

Request / Response cycle:



Controller has 2 objects - request and response as function parameters.

RESTful API:

/ - hierarchy
? - query param
- scrolls to that part

POST - new record entry
PUT - replace complete existing record
PATCH - update something in existing record

ELASTIC SEARCH ->
Uses TF and IDF. and modern uses BM25 algos.